Factories

Generically, the factory pattern refers to a class or module in your application whose sole purpose is to safely and correctly create other objects in your application. Outside of tests, factories are frequently used to encapsulate complex object-creation logic. Inside of Rails tests, factories are used to provide templates for creating valid objects.

Rather than specifying all the test data exactly, the factory tool provides a blueprint for creating a sample instance of your model. When you need data for a specific test, you call a factory method, which gives you an object based on your blueprint. You can override the blueprint to identify any specific attribute values required to make your test work out. Calling the factory method is simple enough to make it feasible to set up a useful amount of data in each test.

The most common factory tools used for Rails testing are factory_bot and its Rails library, factory_bot_rails.[2] [3] The current version as I write this is 4.8.2. Until quite recently this gem was called factory_girl, so you'll likely see many references to it under that name.

Let's talk first about how to set up and use factory_bot, and once you have the basics down I'll talk about how to use it effectively.

Installing factory_bot

To install factory_bot in a Rails project, include the following in the Gemfile in the development and test group:

```
gem 'factory_bot_rails'
```

In a Rails project, factory files are automatically loaded if they are in spec/factories.rb or spec/factories/*.rb. (In Minitest-land, that would be test/factories.rb or test/factories/*.rb.) Factories defined in any other place need to be explicitly required into the program.

There's one optional configuration, which is to place the following line inside the configuration definition in spec/rails_helper.rb or some other file that is loaded as part of the RSpec setup:

2. https://github.com/thoughtbot/factory_bot
3. https://github.com/thoughtbot/factory_bot_rails